Phase 2 Knowledge Extraction: Resource Access Protocols

Daniel Chidi Chimezie

I. Source Scope

Chapter 7 is used as the source text for the extracted knowledge on resource access protocols [1].

II. Extracted Knowledge

A. 7.1 INTRODUCTION

Resource definition. A resource is any software structure that can be used by a process to advance its execution. [1] **Private/shared/exclusive resource.** A resource dedicated to a particular process is said to be private, whereas a resource that can be used by more tasks is called a shared resource. A shared resource protected against concurrent accesses is called an exclusive resource. [1] **Critical section.** A piece of code executed under mutual exclusion constraints is called a critical section. [1] **Blocked task.** A task waiting for an exclusive resource is said to be blocked on that resource, otherwise it proceeds by entering the critical section and holds the resource. [1] **Semaphore.** Operating systems typically provide a general synchronization tool, called a semaphore [Dij68, BH73, PS85], that can be used by tasks to build critical sections. [1]

B. 7.2 THE PRIORITY INVERSION PHENOMENON

Priority inversion. A priority inversion is said to occur in the interval [t3, t6], since the highest-priority task τ1 waits for the execution of lower-priority tasks (τ2 and τ3). [1] **Unbounded duration.** In general, the duration of priority inversion is unbounded, since any intermediate-priority task that can preempt τ3 will indirectly block τ1. [1]

C. 7.3 TERMINOLOGY AND ASSUMPTIONS

Task model. Throughout this chapter, we consider a set of n periodic tasks, τ1, τ2,...,τn, which cooperate through m shared resources, R1, R2,...,Rm. [1] **Nominal and active priority.** Since a protocol modifies the task priority, each task is characterized by a fixed nominal priority Pi (assigned, for example, by the Rate Monotonic algorithm) and an active priority pi (pi ≥ Pi), which is dynamic and initially set to Pi. [1] **Bi.** Bi denotes the maximum blocking time task τi can experience. [1] **zi,k.** zi,k denotes a generic critical section of task τi guarded by semaphore Sk. [1] **Zi,k.** Zi,k denotes the longest critical section of task τi guarded by semaphore Sk. [1] **delta i,k.** δi,k denotes the duration of Zi,k. [1] **Nested critical sections.** The critical sections used by any task are properly nested; that is, given any pair zi,h and zi,k, then either zi,h ⊂ zi,k, zi,k ⊂ zi,h, or zi,h ∩ zi,k = □. [1]

D. 7.4 NON-PREEMPTIVE PROTOCOL

NPP idea. A simple solution that avoids the unbounded priority inversion problem is to disallow preemption during the execution of any critical section. [1] **NPP implementation.** This method, also referred to as Non-Preemptive Protocol (NPP), can be implemented by raising the priority of a task to the highest priority level whenever it enters a shared resource. [1]

E. 7.5 HIGHEST LOCKER PRIORITY PROTOCOL

HLP idea. The Highest Locker Priority (HLP) protocol improves NPP by raising the priority of a task that enters a resource Rk to the highest priority among the tasks sharing that resource. [1] **Immediate Priority Ceiling.** For this reason, this protocol is also referred to as Immediate Priority Ceiling. [1]

F. 7.6 PRIORITY INHERITANCE PROTOCOL

PIP idea. The Priority Inheritance Protocol (PIP), proposed by Sha, Rajkumar and Lehoczky [SRL90], avoids unbounded priority inversion by modifying the priority of those tasks that cause blocking. [1] **Priority inheritance.** In particular, when a task τi blocks one or more higher-priority tasks, it temporarily assumes (inherits) the highest priority of the blocked tasks. [1]

G. 7.7 PRIORITY CEILING PROTOCOL

PCP purpose. The Priority Ceiling Protocol (PCP) was introduced by Sha, Rajkumar, and Lehoczky [SRL90] to bound the priority inversion phenomenon and prevent the formation of

deadlocks and chained blocking. [1] **PCP basic idea.** The basic idea of this method is to extend the Priority Inheritance Protocol with a rule for granting a lock request on a free semaphore. [1] **PCP rule.** To avoid multiple blocking, this rule does not allow a task to enter a critical section if there are locked semaphores that could block it. [1]

H. Chapter 7

SRP. Stack Resource Policy (SRP); Although the first four protocols have been developed under fixed priority assign- ments, some of them have been extended under EDF. [1] **SRP.** 2 The Stack Resource Policy, instead, was natively designed to be applicable under both fixed and dynamic priority assignments. [1] **Preemption level.** Resource Access Protocols 235 PREEMPTION LEVEL Besides a priority pi, a task τi is also characterized by a preemption level πi. [1] **Preemption level.** The preemption level is a static parameter, assigned to a task at its creation time and as- sociated with all instances of that task. [1] **System ceiling.** 238 Chapter 7 SYSTEM CEILING The resource access protocol adopted in the SRP also requires a system ceiling, Πs, defined as the maximum of the current ceilings of all the resources; that is, Πs =m a x k {CRk }. [1] **System ceiling.** Using the definitions introduced in the previous paragraph, this is achieved by the following preemption test: SRP Preemption Test: A task is not permitted to preempt until its priority is the highest among those of all the tasks ready to run, and its preemption level is higher than the system ceiling. [1] **Sharing runtime stack.** (7.18) 7.8.5 SHARING RUNTIME STACK Another interesting implication deriving from the use of the SRP is that it supports stack sharing among tasks. [1] **Schedulability analysis.** We then show how such blocking times can be used in the schedulability analysis to extend the guarantee tests derived for periodic task sets. [1] **Schedulability analysis.** 246 Chapter 7 7.9 SCHEDULABILITY ANALYSIS This section explains how to verify the feasibility of a periodic task set in the presence of shared resources. [1] **Summary.** In this chapter, we describe the main problems that may arise in a uniprocessor system when concurrent tasks use shared resources in exclusive mode, and we present some resource access protocols designed to avoid such problems and bound the maximum blocking time of each task. [1] **Summary.** 7.10 SUMMARY The concurrency control protocols presented in this chapter can be compared with respect to several characteristics. [1]

III. Extracted Equations

7.1 Critical section structure. wait(Sk) ... signal(Sk) — each critical section operating on a resource Rk must begin with a wait(Sk) primitive and end with a signal(Sk) primitive. [1] **7.1 Blocking time.** Bi — Bi denotes the maximum blocking time task τi can experience. [1] **7.1 Generic critical section.** zi,k — zi,k denotes a generic critical section of task τi guarded by semaphore Sk. [1] **7.1 Longest critical section.** Zi,k — Zi,k denotes the longest critical section of task τi guarded by semaphore Sk. [1] **7.1 Duration.** δi,k — δi,k denotes the duration of Zi,k. [1] **7.1 Blocking critical sections.** γi,j = {Zj,k | (Pj < Pi) and (Sk ∈ σi,j)} — γi,j denotes the set of the longest critical sections that can block τi, accessed by the lower priority task τj. [1] **7.2 NPP priority.** pi(Rk) = max_h {Ph} — as soon as a task τi enters a resource Rk, its dynamic priority is raised to the level. [1] **7.3 NPP blocking.** Bi = max_{j,k} {δj,k - 1 | Zj,k ∈ γi} — the maximum blocking time τi can suffer is given by the duration of the longest critical section among those belonging to lower priority tasks. [1] **7.4 HLP priority.** pi(Rk) = max_h {Ph | τh uses Rk} — as soon as a task τi enters a resource Rk, its dynamic priority is raised to the level. [1] **7.5 Resource ceiling.** C(Rk) = max_h {Ph | τh uses Rk} — assigning each resource Rk a priority ceiling C(Rk) (computed off-line) equal to the maximum priority of the tasks sharing Rk. [1] **7.6 HLP blocking.** Bi = max_{j,k} {δj,k - 1 | Zj,k ∈ γi} — Since a task τi can be blocked at most once, the maximum blocking time τi can suffer is given by the duration of the longest critical section among those that can block τi. [1] **7.7 PIP priority inheritance.** pj(Rk) = max{Pj, max_h {Ph | τh is blocked on Rk}} — In general, a task inherits the highest priority of the tasks it blocks. [1] **7.8 PIP bound.** αi = min(li, si) — a task τi can be blocked for at most the duration of αi = min(li, si) critical sections. [1] **7.9 PIP lower task sum.** Bi^l = sum_{j:Pj<Pi} max_k {δj,k - 1 | Zj,k ∈ γi} — for each lower priority task τj that can block τi, sum the duration of the longest critical section among those that can block τi. [1] **7.10 PIP semaphore sum.** Bi^s = sum_{k=1}^{m} max_j {δj,k - 1 | Zj,k ∈ γi} — for each semaphore Sk that can block τi, sum the duration of the longest critical section among those that can block τi. [1] **7.11 PIP upper bound.** Bi = min(Bi^l, Bi^s) — compute Bi as the minimum between Bi^l and Bi^s. [1] **7.12 Semaphore ceiling.** C(Sk) = max_i {Pi | Sk ∈ σi} — for each semaphore Sk we define a ceiling C(Sk) equal to the highest-priority of the

tasks that use Sk. [1] **7.13 PCP ceiling.** C(Sk) = max_i {Pi | Sk ∈ σi} — Each semaphore Sk is assigned a priority ceiling C(Sk) equal to the highest priority of the tasks that can lock it. [1] **7.14 PCP inheritance.** pj(Rk) = max{Pj, max_h {Ph | τh is blocked on Rk}} — In general, a task inherits the highest priority of the tasks blocked by it. [1] **7.15 PCP blocking set.** γi = {Zj,k | (Pj < Pi) and C(Rk) ≥ Pi} — a task τi can only be blocked by critical sections belonging to lower priority tasks with a resource ceiling higher than or equal to Pi. [1]

IV. Extracted Research Questions

RQ1. Which situation is called priority inversion?
A priority inversion is said to occur in the interval [t3, t6], since the highest-priority task τ1 waits for the execution of lower-priority tasks (τ2 and τ3). [1]
RQ2. Which protocols are presented in the chapter?
The rest of this chapter presents the following resource access protocols: Non-Preemptive Protocol (NPP); Highest Locker Priority (HLP), also called Immediate Priority Ceiling (IPC); Priority Inheritance Protocol (PIP); Priority Ceiling Protocol (PCP); Stack Resource Policy (SRP). [1]
RQ3. How does NPP avoid unbounded priority inversion?
A simple solution that avoids the unbounded priority inversion problem is to disallow preemption during the execution of any critical section. [1]
RQ4. How does HLP improve NPP?
The Highest Locker Priority (HLP) protocol improves NPP by raising the priority of a task that enters a resource Rk to the highest priority among the tasks sharing that resource. [1]
RQ5. How does PIP avoid unbounded priority inversion?
The Priority Inheritance Protocol (PIP), proposed by Sha, Rajkumar and Lehoczky [SRL90], avoids unbounded priority inversion by modifying the priority of those tasks that cause blocking. [1]
RQ6. What problems are not prevented by PIP?
Although the Priority Inheritance Protocol bounds the priority inversion phenomenon, the blocking duration for a task can still be substantial because a chain of blocking can be formed. Another problem is that the protocol does not prevent deadlocks. [1]
RQ7. What is the purpose of PCP?
The Priority Ceiling Protocol (PCP) was introduced by Sha, Rajkumar, and Lehoczky [SRL90] to bound the priority inversion phenomenon and prevent the formation of deadlocks and chained blocking. [1]
RQ8. What is the maximum blocking result under PCP?
Under the Priority Ceiling Protocol, a task τi can be blocked for at most the duration of one critical section. [1]

References

[1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. Springer Science+Business Media, LLC, 2011.